MATE (Multi-Agent Tree Engine) System - Architecture Overview

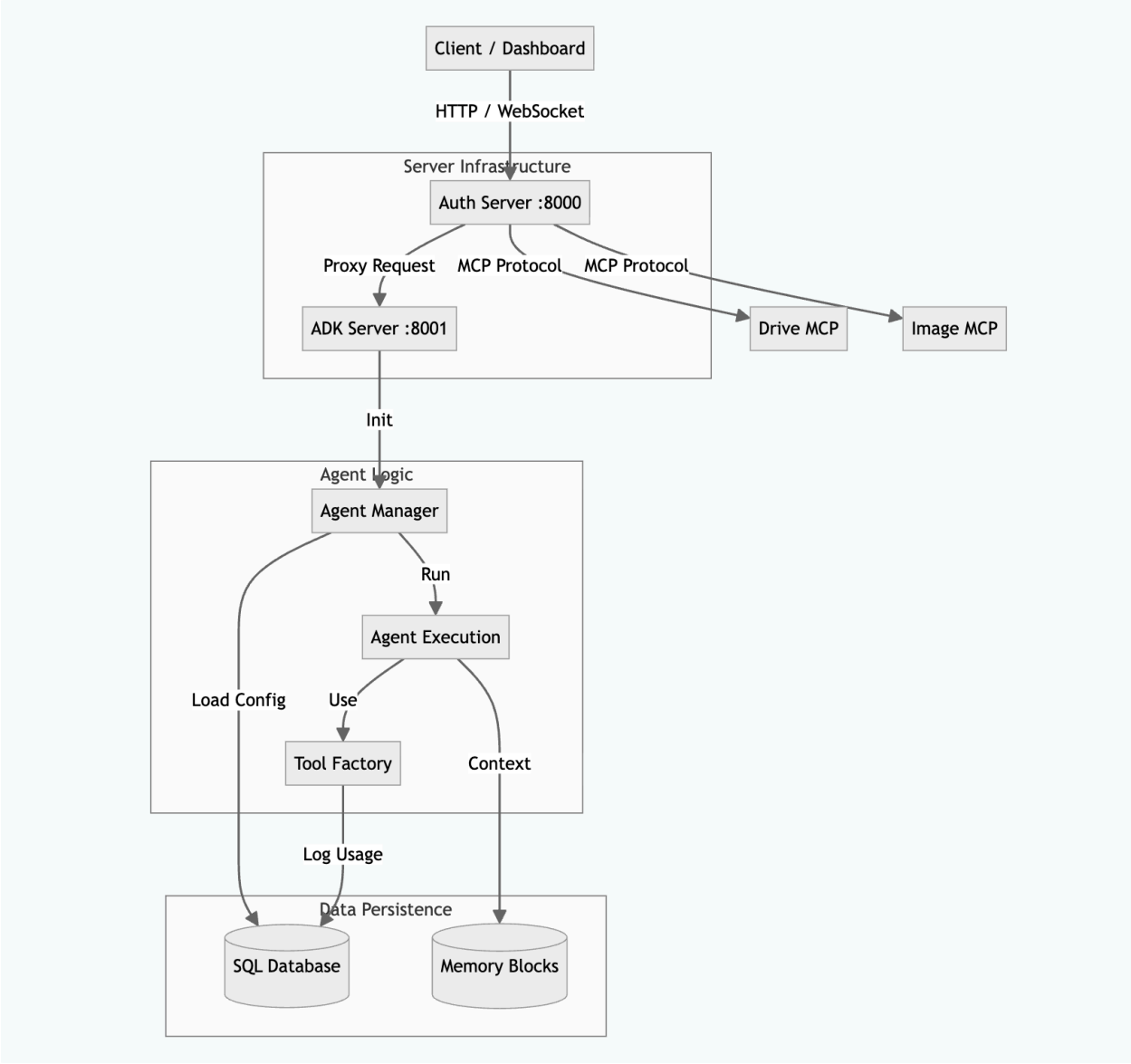
(current state - december 2025)

System Overview

MATE (Multi-Agent Tree Engine) is a multi-agent orchestration system built on Google ADK (Agent Development Kit) that enables database-driven agent configuration, hierarchical agent management, and comprehensive tool integration. The system supports both database-configured and hardcoded agents, with project-level scoping and role-based access control.

High-Level System Diagram

Overview of the two-server architecture (Auth & ADK) and their interaction with the Database, MCP, and Tools.



Architecture Layers

1. Server Layer

Two-Server Architecture:

- **Auth Server** (`auth_server.py`): Port 8000

    - HTTP Basic Auth and Bearer token authentication
    - Proxies all requests to ADK server
    - Hosts dashboard web interface
    - MCP server integration (Image, Google Drive)
    - WebSocket proxying for live agent interactions

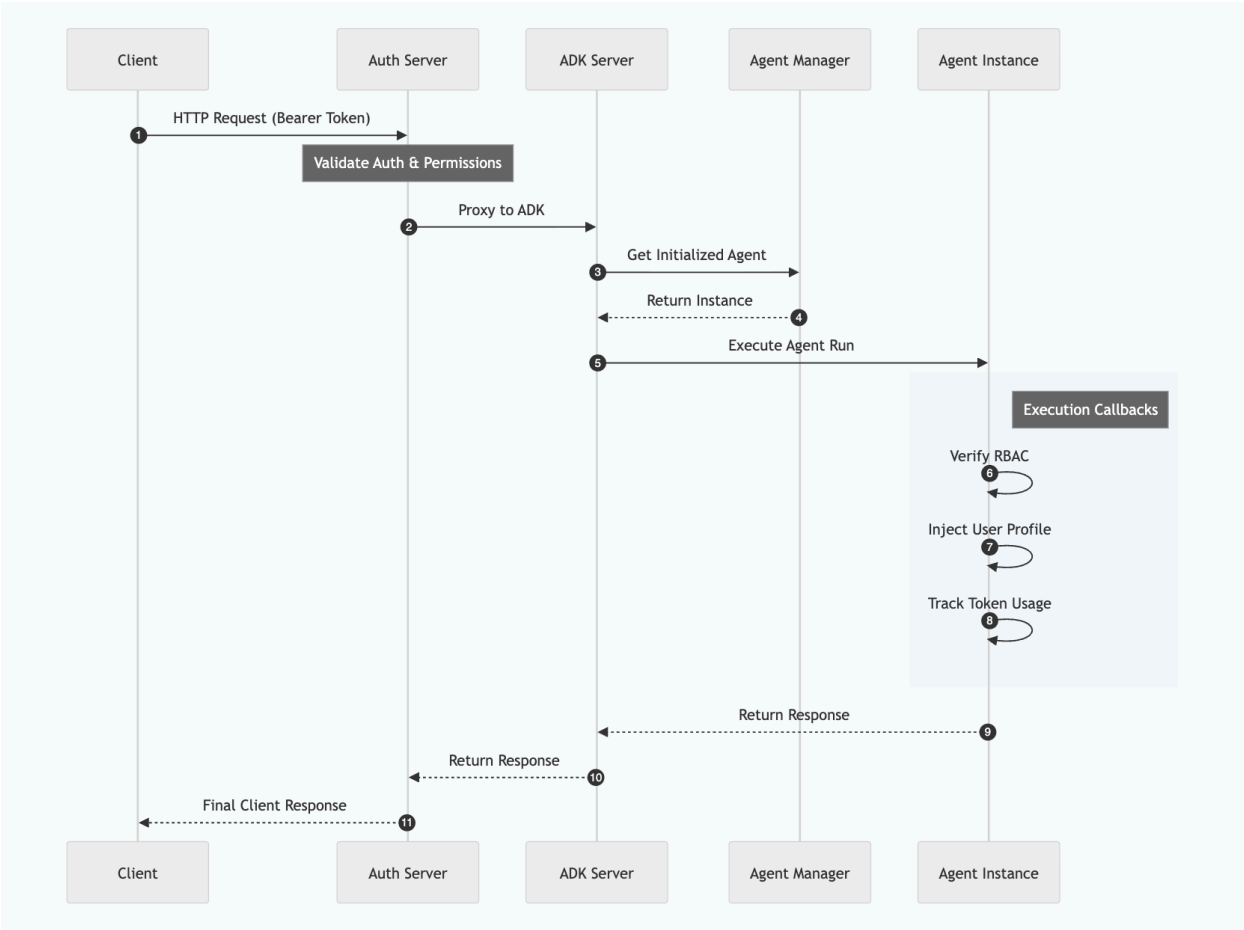
- **ADK Server** (`adk_main.py`): Port 8001

    - Google ADK web server instance
    - Agent execution and session management
    - Artifact, memory, and credential services
    - A2A (Agent-to-Agent) protocol support (optional)

Request Flow:

Client → Auth Server (8000) → ADK Server (8001) → Agent Execution

Sequence of events from user request to agent response, including authentication and callbacks.



2. Agent Management Layer

AgentManager (`shared/utils/agent_manager.py`):

- Database-driven agent initialization
- Hierarchical agent tree building
- Multi-parent agent support
- Agent caching and lifecycle management
- Recursive subagent initialization

Agent Types:

- `llm`: Standard language model agents with tool integration
- `sequential`: Execute subagents in sequence
- `parallel`: Execute subagents concurrently
- `loop`: Execute subagents with configurable iterations

Agent Configuration:

- Stored in `agents_config` table
- Project-scoped via `project_id`
- Supports planners (BuiltInPlanner, PlanReActPlanner)
- Input/output schema validation (JSON Schema → Pydantic)
- Role-based access control via `allowed_for_roles`

Model Support: The system supports multiple model providers with automatic detection and per-agent configuration:

- **LiteLlm (OpenRouter)**: Multi-model support via OpenRouter API
    - Models prefixed with `openrouter/` (e.g., `openrouter/deepseek/deepseek-chat-v3.1`, `openrouter/anthropic/claude-3.5-sonnet`)
    - Uses `OPENROUTER_API_KEY` and `OPENROUTER_BASE_URL` environment variables
    - Supports 100+ models from various providers (Anthropic, OpenAI, Meta, etc.)
    - Default base URL: `https://openrouter.ai/api/v1`
- **Gemini**: Google's Gemini models via direct API
    - Direct integration with Google Gemini API
    - Uses `GOOGLE_API_KEY` environment variable
    - Supported models: `gemini-1.5-flash`, `gemini-1.5-pro`, `gemini-2.5-flash`, etc.
    - Default model: `gemini-1.5-flash` (configurable via `GEMINI_MODEL`)

Model Configuration:

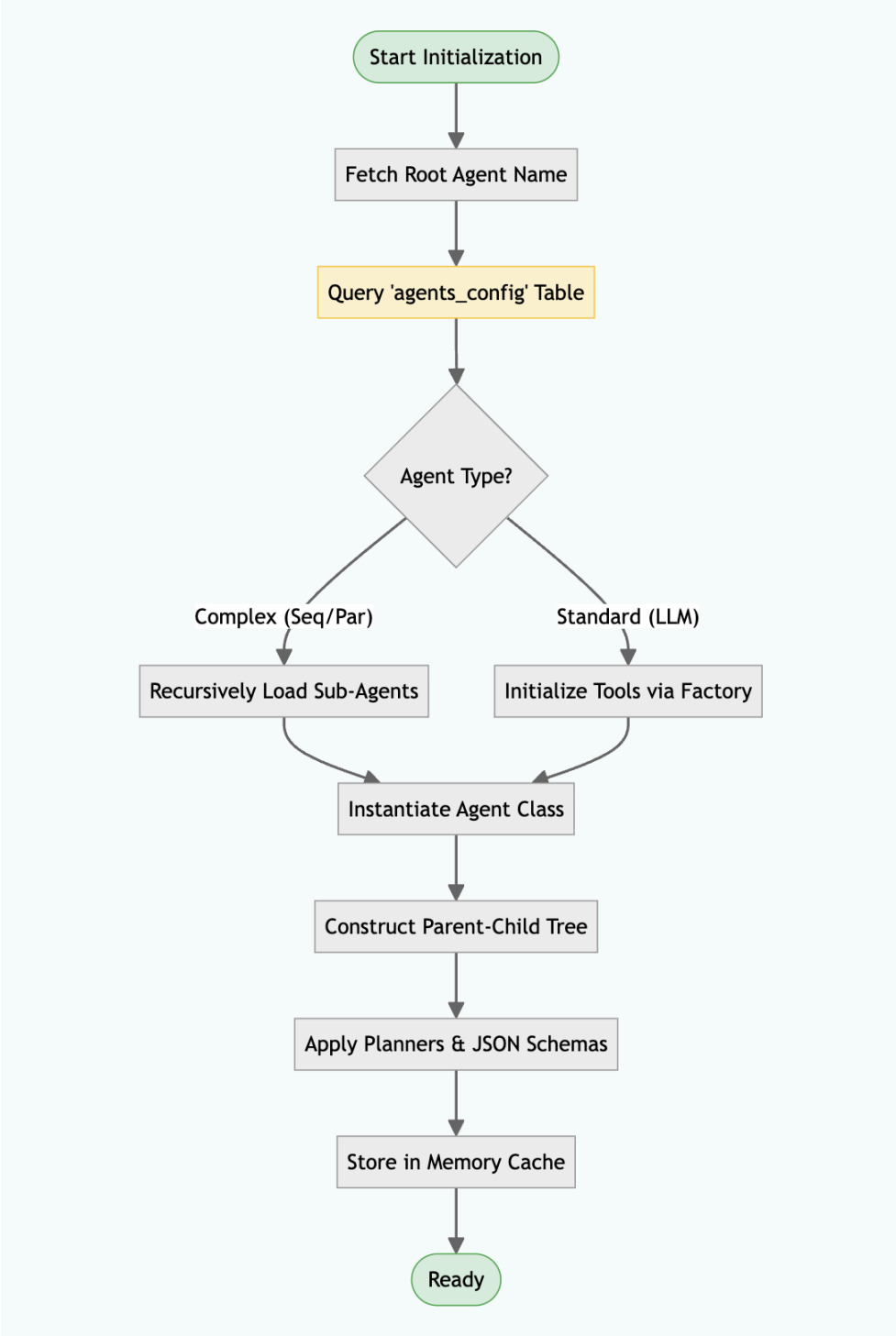
- **Per-Agent Models**: Each agent can specify `model_name` in `agents_config` table
- **Default Model**: Controlled by `MODEL_TYPE` environment variable (`litellm` or `gemini`)
- **Auto-Detection**: `create_model()` function automatically detects model type based on name prefix
- **Custom Configuration**: Can override API keys and base URLs per agent via `create_model()` parameters
- **Model Factory**: Centralized model creation via `shared/utils/utils.py::create_model()`

Example Model Names:

- OpenRouter: `openrouter/deepseek/deepseek-chat-v3.1`, `openrouter/anthropic/claude-4.5-sonnet`, `openrouter/meta-llama/llama-3.1-70b-instruct`
- Gemini: `gemini-2.5-flash`, `gemini-3.0-pro`, `gemini-2.5-flash-lite`

Agent Initialization Process

How the AgentManager loads configuration from the DB and recursively builds the agent tree.



3. Tool Integration Layer

ToolFactory (`shared/utils/tools/tool_factory.py`):

- Centralized tool creation system
- Supports multiple tool types:
  - MCP tools (Model Context Protocol)
  - Google Search/Drive integration
  - CV analysis tools
  - Image generation tools
  - File search (vector stores)
  - User profile tools
  - Custom function tools
  - Memory integration

Tool Configuration:

- Configured via `tool_config` JSON field
- Auto-configured File Search stores from database
- Automatic user profile tool injection

4. Database Layer

DatabaseClient (`shared/utils/database_client.py`):

- SQLAlchemy ORM abstraction
- Multi-database support (PostgreSQL, MySQL, SQLite)
- Connection pooling and session management
- Automatic migration execution on startup

Core Tables:

- `projects`: Project scoping for agents
- `agents_config`: Agent configurations with JSON fields
- `users`: User management and RBAC
- `token_usage_logs`: Token consumption tracking
- `migrations`: Migration version tracking

Migration System:

- Database-specific migration files
- Automatic execution on server startup
- Version tracking with checksums

- Rollback support
- Located in `shared/sql/migrations/{db_type}/`

5. Authentication & Authorization

RBAC System:

- User roles stored in `users` table (JSON array)
- Agent-level access control via `allowed_for_roles`
- RBAC callbacks on agent execution
- User profile injection for personalization

Authentication Methods:

- HTTP Basic Auth (configurable credentials)
- Bearer token authentication
- Session management with logout support

6. Dashboard & Management

DashboardServer (`shared/utils/dashboard/dashboard_server.py`):

- Web-based management interface
- Agent CRUD operations
- User management
- Usage analytics and token tracking
- Migration management
- Server control (start/stop/restart ADK server)

Features:

- Project-based agent filtering
- Agent import/export (JSON)
- Visual agent hierarchy
- Real-time usage statistics
- Token usage logs with pagination

7. MCP Integration

Built-in MCP Servers:

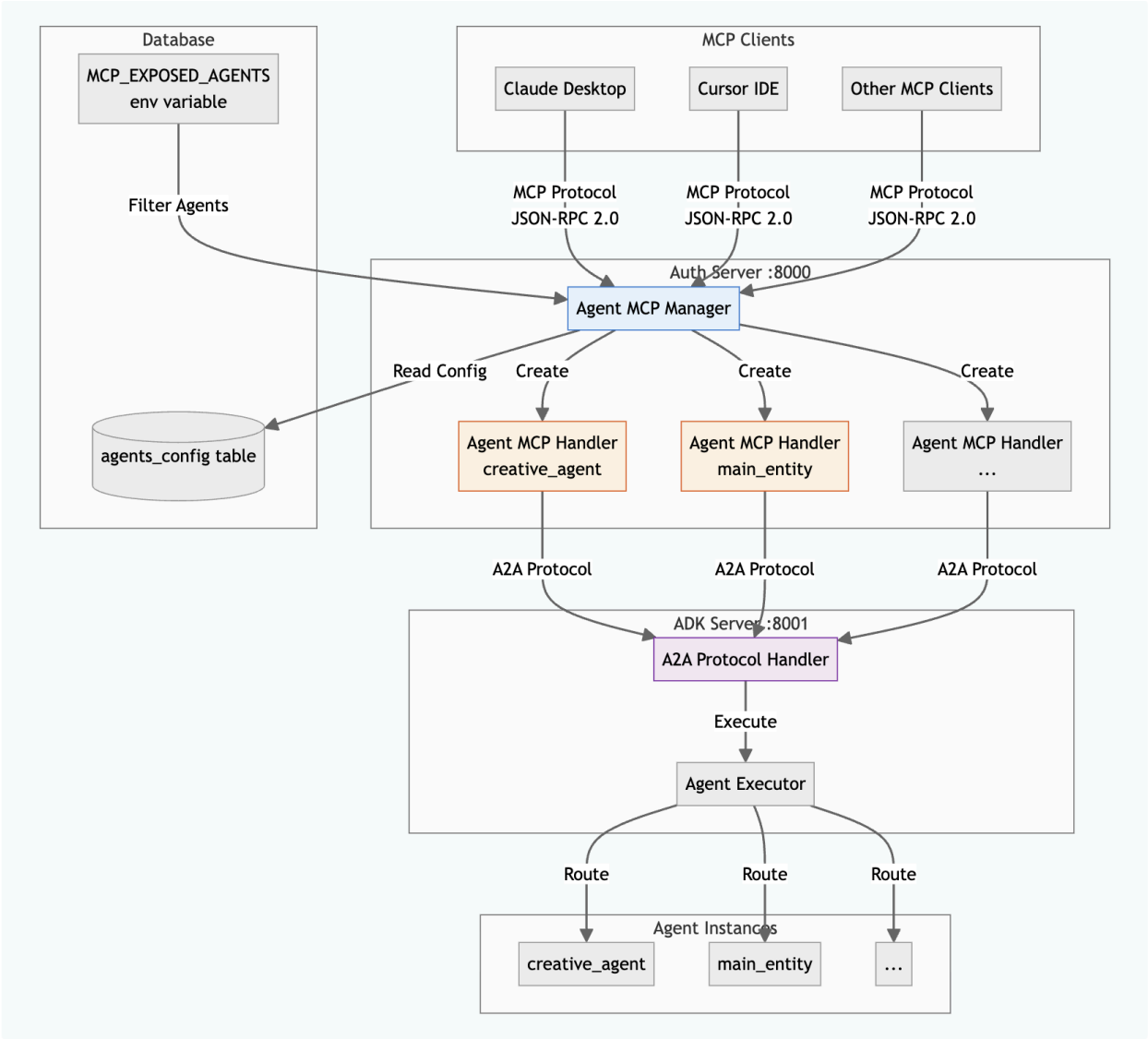
- Image MCP Server (`/images/mcp`): DALL-E, DALL-E 3, GPT Image 1, Nano Banana (Gemini 2.0 Flash Exp)...
- Google Drive MCP Server (`/gdrive/mcp`): File operations

- Exposed at `/mcp` endpoints
- SSE (Server-Sent Events) support
- CORS-enabled for web clients

Agent MCP Servers (Dynamic):

Agents can be dynamically exposed as MCP servers, allowing any MCP-compatible client (Claude Desktop, Cursor IDE, etc.) to interact with agents through the standard MCP protocol.

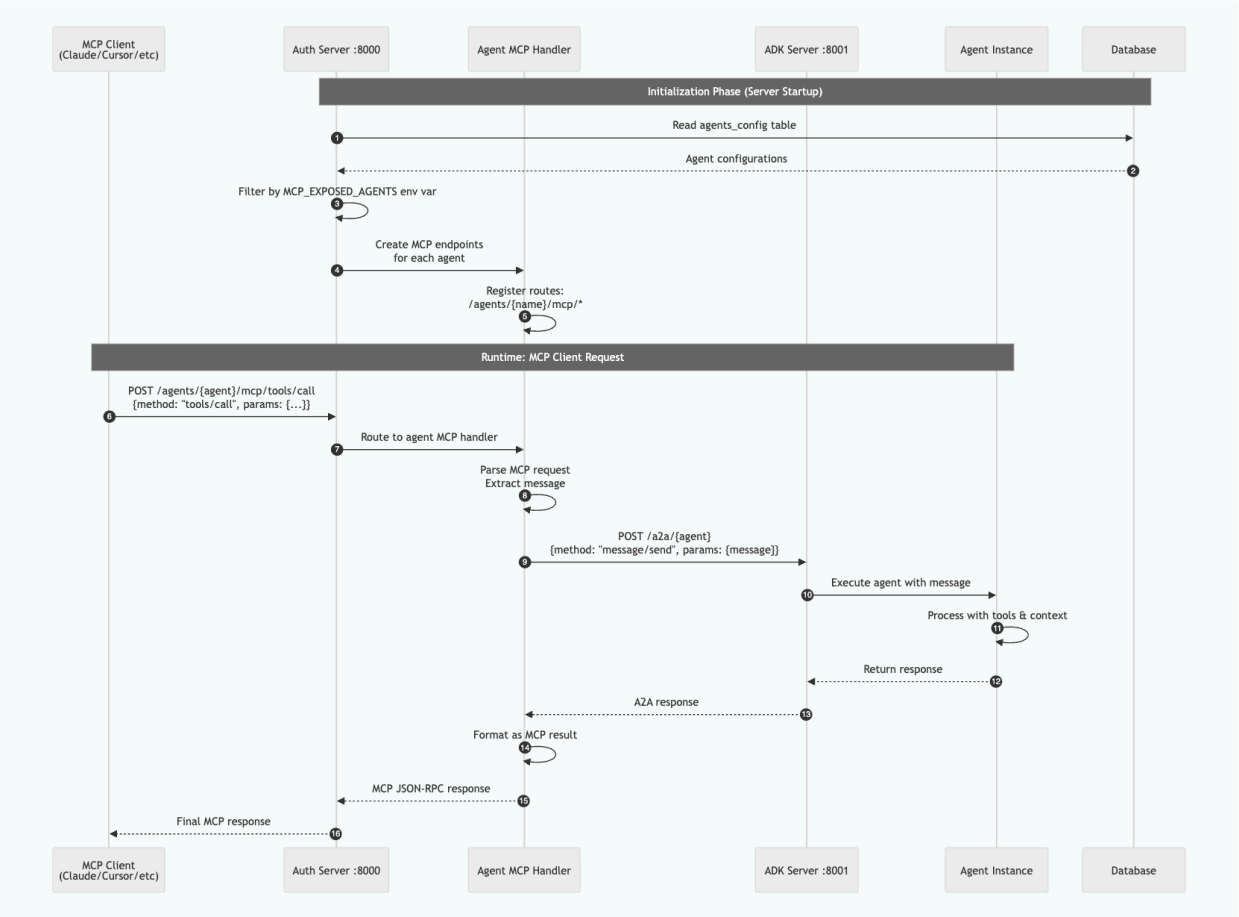
Architecture:



- **Initialization Phase (Server Startup):**

- `AgentMCPManager` reads `agents_config` table from database
- Filters agents based on `MCP_EXPOSED_AGENTS` environment variable
- Creates `AgentMCPServer` instance for each enabled agent
- Registers MCP endpoints: `/agents/{agent_name}/mcp/*`

- **Runtime Flow:**



MCP Client → Auth Server (8000) → Agent MCP Handler → ADK Server (8001) → Agent Execution

- **Protocol Translation:**

  - MCP JSON-RPC 2.0 protocol → A2A (Agent-to-Agent) protocol → Agent execution
  - Each agent exposes one tool: `call_{agent_name}_agent`
  - Tool accepts `message` (required) and `session_id` (optional) parameters

Components:

- **AgentMCPManager** (`shared/utils/mcp/agent_mcp_manager.py`):
    - Manages dynamic creation of agent MCP servers
    - Reads configuration from database and environment
    - Creates and registers MCP endpoints at startup
- **AgentMCPServer** (`shared/utils/mcp/agent_mcp_server.py`):
    - Creates MCP endpoints for individual agents
    - Registers standard MCP routes (health, initialize, tools/list, tools/call, sse)
    - Handles CORS and authentication
- **AgentMCPProtocolHandler** (`shared/utils/tools/agent_mcp_protocol_handler.py`):
    - Translates MCP protocol requests to A2A protocol
    - Formats agent responses as MCP JSON-RPC results
    - Handles error responses and protocol compliance

Configuration:

- **Environment Variable:** `MCP_EXPOSED_AGENTS=agent1,agent2,agent3`
    - Comma or space-separated list of agent names
    - Only listed agents are exposed as MCP servers
    - Agent names must match exactly (case-sensitive) with database entries
- **Database Requirements:**
    - Agents must exist in `agents_config` table
    - Agents must have `disabled = false`
    - Agent descriptions are used as MCP server descriptions

MCP Endpoints (per agent):

- `GET /agents/{agent_name}/mcp/health` - Health check (no auth)
- `GET /agents/{agent_name}/mcp` - Server info
- `POST /agents/{agent_name}/mcp/initialize` - MCP protocol initialization
- `POST /agents/{agent_name}/mcp/tools/list` - List available tools
- `POST /agents/{agent_name}/mcp/tools/call` - Execute tool calls
- `GET /agents/{agent_name}/mcp/sse` - Server-Sent Events stream
- `OPTIONS /agents/{agent_name}/mcp` - CORS preflight

Request Flow:

1. MCP Client sends: `POST /agents/{agent}/mcp/tools/call`

```
{
    "jsonrpc": "2.0",
    "method": "tools/call",
    "params": {
      "name": "call_{agent}_agent",
      "arguments": {"message": "Hello"}
    }
  }
```

2. AgentMCPProtocolHandler parses MCP request

3. Handler calls ADK Server via A2A:

```
POST /a2a/{agent}
{
  "jsonrpc": "2.0",
  "method": "message/send",
  "params": {
    "message": {
      "messageId": "...",
      "role": "user",
      "parts": [{"kind": "text", "text": "Hello"}]
    }
  }
}
```

4. ADK Server executes agent

5. Handler formats response as MCP result:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "content": [{"type": "text", "text": "Agent response"}]
  }
}
```

Benefits:

- **Standard Protocol**: Agents accessible via industry-standard MCP protocol
- **Client Compatibility**: Works with any MCP-compatible client

- **Dynamic Configuration**: No code changes needed - configure via database and env vars
- **Protocol Abstraction**: MCP clients don't need to know about A2A or internal architecture
- **Scalability**: Each agent gets its own MCP endpoint automatically

Use Cases:

- Claude Desktop integration for agent access
- Cursor IDE integration for development workflows
- Custom MCP clients for specific use cases
- Multi-client access to the same agents

8. Callbacks & Monitoring

Callbacks:

- Token usage tracking (`token_usage_callback.py`)
- RBAC enforcement (`rbac_callback.py`)
- User profile injection (`user_profile_callback.py`)
- Model name capture for analytics
- Combined callbacks for parallel agent compatibility

Monitoring:

- Prometheus metrics integration (automatic HTTP metrics via `prometheus-fastapi-instrumentator`)
    - Exposes `/metrics` endpoint with standard HTTP metrics (request count, latency, status codes)
    - Automatic instrumentation of all FastAPI routes
    - No custom metrics currently implemented
- Token usage logging to database
- Request/response tracking
- Error logging with status codes

9. Dynamic Memory Management

Bootstrap Pattern: The system implements a "bootstrap" pattern where agents use minimal hardcoded instructions and dynamically load detailed instructions from memory blocks at runtime. This enables zero-deployment updates and efficient context management.

Architecture:

- **Minimal Bootstrap Instructions**: Agents start with minimal identity and bootstrap protocol
- **Memory Integration**: Instructions stored in shared memory blocks
- **Dynamic Loading**: Agents search and load instructions on-demand via tools
- **Namespace Strategy**: Organized by `system_instruction_shared_*` (all agents) and `system_instruction_<agentName>_*` (agent-specific)

Memory Tools (`shared/utils/tools/memory_blocks_tools.py`):

- `list_shared_blocks`: Search memory blocks by label/prefix/values
- `get_shared_block`: Retrieve specific memory block
- `modify_shared_block`: Update memory block content
- `create_shared_block`: Create new memory blocks
- `delete_shared_block`: Remove memory blocks

Memory Block Types:

- **System Instructions** (Eager Loading): Routing rules, escalation protocols, output formats - loaded at startup via bootstrap protocol
- **User Memory** (Eager Loading): Per-user preferences and context (`human_<current_user>`) - loaded at startup
- **Lazy-Loaded Blocks** (On-Demand): Large schemas, specialized formats, rarely-used protocols - loaded only when specific prompts/conditions are detected
    - Examples: `smart_object_output_format_json`, `complex_report_schema`, `specialized_protocol_*`

Lazy Loading for Specific Prompts: For large or rarely-used instructions (e.g., complex JSON schemas, specialized formats), the system supports lazy loading based on user prompts or conditions:

- **Purpose**: Avoid loading large memory blocks unless needed, reducing token usage and improving response time
- **Naming Convention**: Use distinct prefixes (e.g., `smart_object_output_format_*`) to differentiate from core `system_instruction_*` blocks
- **Trigger-Based Loading**: Agents detect specific keywords or conditions in user requests and load relevant blocks on-demand

Lazy Loading Protocol:

```
1. Agent receives user request

2. Agent analyzes request for lazy-load triggers:

   - Keywords: "visualization", "frontend data", "smart object", etc.

   - Conditions: Specific task types or output formats needed

3. Agent calls list_shared_blocks(label="<specific_block_label>")

4. Agent loads and applies the schema/instructions

5. Agent formats response using loaded schema
```

Example Use Cases:

- **Smart Object JSON Schema**: Load `smart_object_output_format_json` only when user requests visualization or frontend data
- **Complex Output Formats**: Load specialized formatting instructions only when specific output types are requested
- **Rarely-Used Protocols**: Load detailed protocols for edge cases only when triggered

Example Lazy Loading Flow:

```
User: "Show me the org chart visualization"
↓
Agent detects keywords: "visualization", "org chart"
↓
Agent calls: list_shared_blocks(label="smart_object_output_format_json")
↓
Agent receives JSON schema for smart object format
↓
Agent generates response formatted according to loaded schema
```

Bootstrap Instruction Example:

LAZY LOADING PROTOCOL:

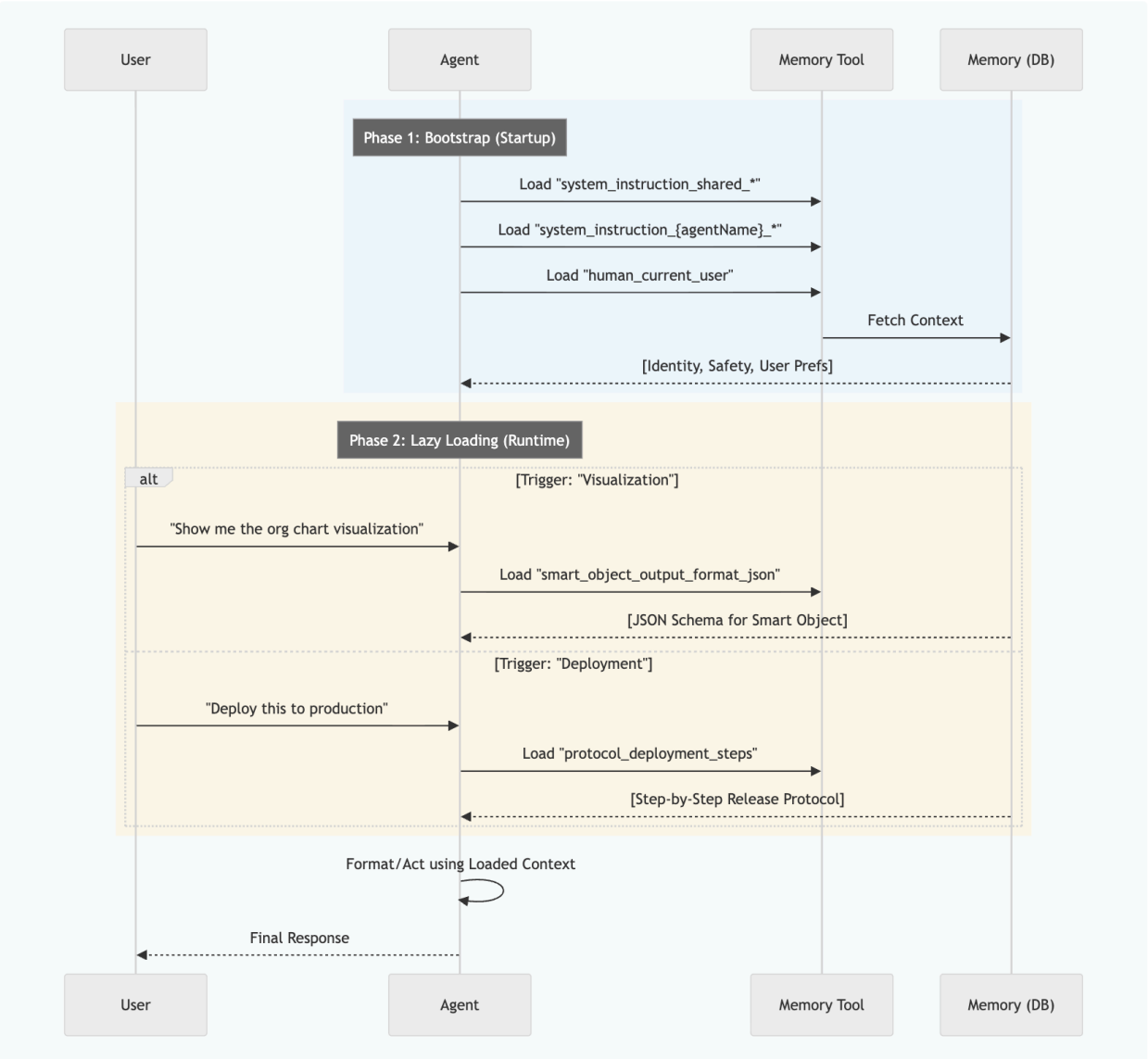
```
- IF the user asks for "visualization", "frontend data", or "smart object":

- THEN call list_shared_blocks(label="smart_object_output_format_json")

- AND use that schema to format your response.
```

Bootstrap Protocol Flow:

```
1. Agent starts with minimal instruction

2. Agent calls list_shared_blocks(label_search="system_instruction_shared_")

3. Agent calls list_shared_blocks(label_search="system_instruction_<role>_")

4. Agent calls list_shared_blocks(label="human_current_user")

5. Agent loads all found blocks and treats as system instructions

6. Agent executes with dynamically loaded context
```

Dynamic Bootstrap Memory Flow

The "Zero-Deployment" update strategy where agents fetch instructions at runtime.



Benefits:

- **Zero Deployment Updates**: Change agent behavior by editing memory blocks
- **Context Efficiency**: Load instructions only when needed
- **Shared Knowledge**: Multiple agents can subscribe to same memory blocks
- **Separation of Concerns**: Instructions separated from code
- **Runtime Adaptation**: Agents can update user memory automatically

Configuration:

- Enabled via `tool_config` with `memory_blocks` configuration
- Project-scoped memory blocks for multi-tenancy

Key Technologies

- **Google ADK**: Agent framework and execution engine
- **FastAPI**: Web framework for both servers
- **SQLAlchemy**: ORM for database operations
- **Pydantic**: Schema validation and model generation
- **MCP**: Model Context Protocol for tool integration
- **Prometheus**: Automatic HTTP metrics collection (via prometheus-fastapi-instrumentator)
- **Jinja2**: Template engine for dashboard
- **WebSockets**: Live agent interactions

Deployment Architecture

Services:

- Artifact Service: Local folder, S3, or Supabase
- Memory Service: Database-backed (DBMemoryService)
- Session Service: Database-backed (DatabaseSessionService)
- Credential Service: Database-backed (DBCredentialService)

Environment Configuration:

- Database: `DATABASE_URL` or individual DB params
- Model: `MODEL_TYPE` (litellm/gemini), API keys
- Auth: `AUTH_USERNAME`, `AUTH_PASSWORD`
- Services: `ARTIFACT_SERVICE`, `MEMORY_SERVICE`, `CREDENTIAL_SERVICE`

Docker Support:

- Docker Compose configuration
- Volume mounting for development
- Environment variable injection
- Health checks and monitoring

Design Patterns

- **Factory Pattern**: ToolFactory for tool creation
- **Manager Pattern**: AgentManager for agent lifecycle
- **Proxy Pattern**: Auth server proxying to ADK server
- **Callback Pattern**: Event-driven callbacks for monitoring
- **Repository Pattern**: DatabaseClient abstraction
- **Service Registry**: ADK service registry for extensibility

Scalability Considerations

- Agent caching reduces database queries
- Connection pooling for database operations
- Project-level scoping enables multi-tenancy
- Stateless agent execution (session-based)
- Horizontal scaling via multiple ADK server instances
- Database migrations support schema evolution